Teoria współbieżności

Sprawozdanie z laboratorium 1

Adrian Mikoda

12.10.2026 r.

Spis treści

1. Dane techniczne . . . 2
2. Treści zadań . . . 2
   2.1. Zadanie 1 . . . 2
   2.2. Zadanie 2 . . . 2
   2.3. Zadanie 3 . . . 2
3. Realizacja ćwiczenia . . . 2
   3.1. Zadanie 1 . . . 2
      3.1.1. Punkty 1.1 oraz 1.2 – program bez synchronizacji . . . 2
      3.1.2. Punkt 1.3 – mechanizm własny (zamek Petersona) . . . 4
      3.1.3. Klasa główna programu . . . 5
   3.2. Zadanie 2 . . . 7
4. Wyniki . . . 7
   4.1. Zadanie 1 . . . 7
   4.2. Zadanie 2 . . . 8
5. Wnioski . . . 9
Bibliografia . . . 10

1. Dane techniczne

- System operacyjny: `Windows 11 Home`
- Procesor: `AMD Ryzen 5 7530U (2.0 - 4.5 GHz)`
- Pamięć RAM: `16GB LPDDR4 (2133 MHz)`
- SDK: `Microsoft OpenJDK 25.0.4`
- Opcje JVM: `-Xint` (wyłączony kompilator JIT),
- Wizualizacja wyników: `Python 3.13.5`
- Użyte biblioteki: `matplotlib 3.11.2, numpy 2.5.3`

2. Treści zadań

2.1. Zadanie 1

1. Napisać program (szkielet), który uruchamia 2 wątki, z których jeden zwiększa wartość zmiennej całkowitej o 1, drugi wątek zmniejsza wartość o 1. Zakładając, że na początku wartość zmiennej `Counter` była 0, chcielibyśmy wiedzieć, jaka będzie wartość tej zmiennej po wykonaniu 10000 operacji zwiększania i zmniejszania przez obydwa wątki.
2. Na podstawie 100 wykonań programu z p. 1.1 stworzyć histogram końcowych wartości zmiennej `Counter`.
3. Spróbować wprowadzić mechanizm do programu z p. 1, który zagwarantowałby przewidywalną końcową wartość zmiennej `Counter`. Nie używać żadnych systemowych mechanizmów (np. `synchronized`), tylko swój autorski.

2.2. Zadanie 2

W systemie działa N wątków, które dzielą obiekt licznika (początkowy stan licznika = 0). Każdy wątek wykonuje w pętli 5 razy inkrementację licznika. Zakładamy, że inkrementacja składa się z sekwencji trzech instrukcji: `read`, `inc`, `write` (odczyt z pamięci, zwiększenie o 1, zapis do pamięci). Wątki nie są synchronizowane.

1. Jaka jest teoretycznie najmniejsza wartość licznika po zakończeniu działania wszystkich wątków i jaka kolejność instrukcji (przeplot) do niej prowadzi?
2. Spróbować znaleźć dowód, że będzie to zawsze najmniejsza wartość.

2.3. Zadanie 3

Napisać sprawozdanie z realizacji pp. 1–2, z argumentacją i interpretacją wyników.

3. Realizacja ćwiczenia

3.1. Zadanie 1

Zadanie zrealizowano w języku Java. Wspólny obiekt klasy `Counter` przechowuje wartość całkowitą (początkowo 0) i udostępnia metody `inc()` oraz `dec()`.

3.1.1. Punkty 1.1 oraz 1.2 – program bez synchronizacji

Uruchamiane są dwa wątki: `IThread` wykonujący `inc()` oraz `DThread` wykonujący `dec()`. Każdy z nich wykonuje po $n = 10000$ operacji na wspólnym liczniku.

Eksperyment powtórzono 100 razy, a wyniki zapisano do pliku `race_without_lock.txt`. Implementacje klas `Counter`, `IThread` oraz `DThread` przedstawiono we fragmentach kodu nr 1 oraz 2 .

```
1  class Counter {
2      private int _val;
3      public Counter(int n) {_val = n;}
4      public void inc() {_val++;}
5      public void dec() {_val--;}
6      public int value() {return _val;}
7  }
```

Fragment kodu 1: Klasa `Counter`

```
1  class IThread extends Thread {
2      Counter counter;
3      int n;
4
5      public IThread(Counter counter, int n) {
6          this.counter = counter;
7          this.n = n;
8      }
9
10     @Override
11     public void run() {
12         for (int i = 0; i < n; i++) {
13             counter.inc();
14         }
15     }
16 }
17
18 class DThread extends Thread {
19     Counter counter;
20     int n;
21
22     public DThread(Counter counter, int n) {
23         this.counter = counter;
24         this.n = n;
25     }
26
27     @Override
28     public void run() {
29         for (int i = 0; i < n; i++) {
30             counter.dec();
31         }
32     }
33 }
```

Fragment kodu 2: Klasa `DThread` oraz `IThread`

Operacje `inc()` oraz `dec()` na liczniku nie są atomowe: składają się z odczytu (`read`), zwiększenia (`inc`) i zapisu (`write`), co powoduje wyścig między 2 różnymi wątkami.

3.1.2. Punkt 1.3 – mechanizm własny (zamek Petersona)

Aby zagwarantować przewidywalny wynik, zaimplementowano algorytm Petersona [1] wzajemnego wykluczania dla dwóch wątków w klasie `PetersonLock`. Zamek składa się z:

- pól `want0` i `want1` wskazujących, czy dany wątek chce wejść do sekcji krytycznej,
- zmiennej `turn` wskazującej, któremu wątkowi ustąpiono pierwszeństwa.

Wątek o identyfikatorze `id` w metodzie `lock(id)` ustawia odpowiednie `want = true`, następnie oddaje pierwszeństwo drugiemu wątkowi (`turn = other`) i oczekuje aktywnie, dopóki drugi wątek chce wejść do sekcji krytycznej i ma pierwszeństwo. Metoda `unlock(id)` ustawia odpowiednie `want = false`. Pola `want` i `turn` zadeklarowano jako `volatile`, aby zapewnić widoczność zmian między wątkami i uniemożliwić zmianę kolejności ich odczytów i zapisów, co jest gwarantowane przez model pamięci Javy.

Każda operacja `inc()` (wątek o identyfikatorze 0, `IThreadLock`) oraz `dec()` (wątek o identyfikatorze 1, `DThreadLock`) jest otoczona wywołaniami `lock` i `unlock`. Każdy z nich wykonuje po $n = 10000$ operacji na wspólnym liczniku. Eksperyment powtórzono 100 razy, a wyniki zapisano do pliku `race_with_lock.txt`.

Implementacje klas `PetersonLock`, `IThreadLock` oraz `DThreadLock` przedstawiono we fragmentach kodu nr 3 oraz 4.

```
1  class PetersonLock {
2      volatile boolean want0 = false, want1 = false;
3      volatile int turn;
4
5      private boolean wantOther(int id) {
6          return id == 0 ? want1 : want0;
7      }
8
9      public void lock(int id) {
10         int other = 1 - id;
11         if (id == 0) {want0 = true;} else {want1 = true;}
12         turn = other;
13         while (wantOther(id) && turn == other) {}
14     }
15
16     public void unlock(int id) {
17         if (id == 0) {want0 = false;} else {want1 = false;};
18     }
19 }
```

Fragment kodu 3: Klasa `PetersonLock`

```
class IThreadLock extends IThread {
    PetersonLock petersonLock;

    public IThreadLock(Counter counter, int n, PetersonLock petersonLock) {
        super(counter, n);
        this.petersonLock = petersonLock;
    }

    @Override
    public void run() {
        for (int i = 0; i < n; i++) {
            petersonLock.lock(0);
            counter.inc();
            petersonLock.unlock(0);
        }
    }
}


class DThreadLock extends DThread {
    PetersonLock petersonLock;

    public DThreadLock(Counter counter, int n, PetersonLock petersonLock) {
        super(counter, n);
        this.petersonLock = petersonLock;
    }

    @Override
    public void run() {
        for (int i = 0; i < n; i++) {
            petersonLock.lock(1);
            counter.dec();
            petersonLock.unlock(1);
        }
    }
}
```

Fragment kodu 4: Klasa `IThreadLock` oraz `DThreadLock`

3.1.3. Klasa główna programu

Wszystkie klasy opisane we fragmentach kodu nr 1, 2, 3 oraz 4 wykorzystano w głównej części programu zgodnie z implementacją przedstawioną we fragmencie kodu nr 5.

```
public class Race {
    private static int getCounterValue(Counter counter, DThread dThread, IThread
iThread) {
        dThread.start();
        iThread.start();
        try {
            dThread.join();
            iThread.join();
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }

        return counter.value();
```

```
13      }
14
15      public static void main(String[] args) {
16          String path_without_lock = "../race_without_lock.txt";
17          String path_with_lock = "../race_with_lock.txt";
18
19          try {
20              Files.deleteIfExists(Paths.get(path_without_lock));
21              Files.deleteIfExists(Paths.get(path_with_lock));
22          } catch (Exception e) {
23              e.printStackTrace();
24          }
25
26          for (int i = 0; i < 100; i++) {
27              Counter counter = new Counter(0);
28              DThread dThread = new DThread(counter, 10000);
29              IThread iThread = new IThread(counter, 10000);
30              String content = getCounterValue(counter, dThread, iThread) +
System.lineSeparator();
31              try {
32              Files.write(Paths.get(path_without_lock), content.getBytes(),
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
33              } catch (Exception e) {e.printStackTrace();}
34          }
35
36          for (int i = 0; i < 100; i++) {
37              Counter counter = new Counter(0);
38              PetersonLock petersonLock = new PetersonLock();
39              DThreadLock dThreadLock = new DThreadLock(counter, 10000, petersonLock);
40              IThreadLock iThreadLock = new IThreadLock(counter, 10000, petersonLock);
41
42              String content = getCounterValue(counter, dThreadLock, iThreadLock) +
System.lineSeparator();
43              try {
44                  Files.write(Paths.get(path_with_lock), content.getBytes(),
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
45              } catch (Exception e) {e.printStackTrace();}
46          }
47      }
48  }
```

Fragment kodu 5: Klasa główna programu

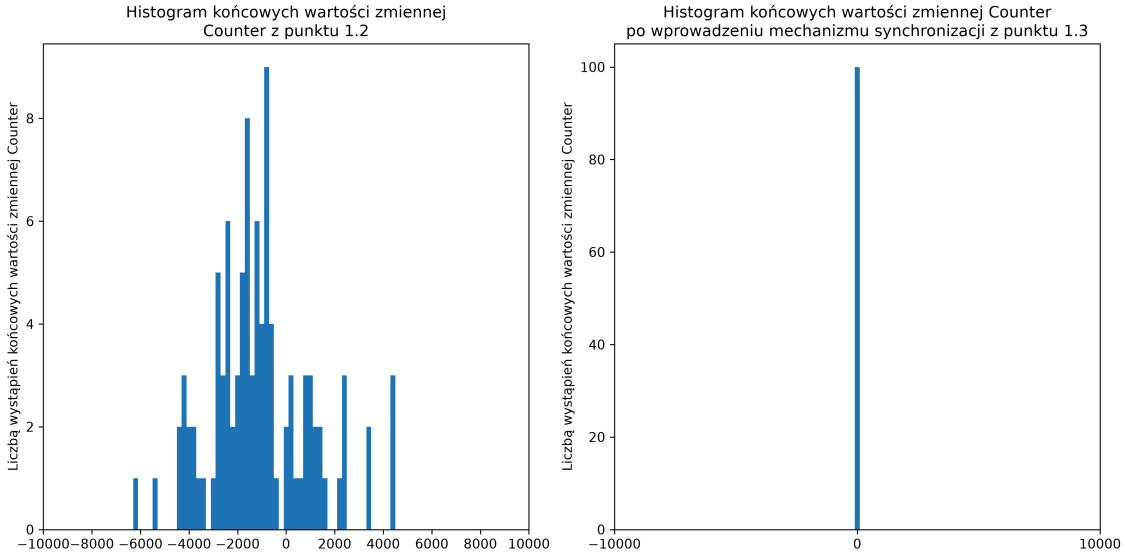
3.2. Zadanie 2

Odpowiedź oraz dowód na minimalną wartość końcową licznika przedstawiono w sekcji 4.2 sprawozdania.

4. Wyniki

4.1. Zadanie 1

Otrzymane wyniki eksperymentu przedstawiono w postaci 2 histogramów na rysunku nr 1. Oba histogramy wygenerowano z rozdzielczością 100 przedziałów i zakresem osi x równym $[-10000, 10000]$.



Rysunek 1: Histogramy wartości końcowych zmiennej `Counter`

Na histogramie z punktu 1.2 (bez synchronizacji) wartości końcowe są rozproszone w szerokim zakresie. Na histogramie z punktu 1.3 (z zamkiem Petersona) wszystkie wyniki są równe oczekiwanej wartości 0. Świadczy to o występowaniu wyścigu (*race condition*), nieatomowości operacji `inc()` oraz `dec()` w wersji niesynchronizowanej, a także o poprawności rozwiązania problemu algorytmem Petersona.

Statystyki końcowych wartości zmiennej `Counter` zestawiono w tabeli nr 1.

Statystyka	**Bez synchronizacji**	**Z zamkiem Petersona**
Liczba wykonań	100	100
Wykonania z wynikiem 0	0	100
Minimum	-6134	0
Maksimum	4484	0
Średnia	$-1004,42$	0,0
Mediana	$-1175,0$	0,0

Tabela 1: Statystyki końcowych wartości zmiennej `Counter`

W wersji bez synchronizacji żadne ze 100 wykonań nie dało poprawnego wyniku 0. Wyniki mieszczą się w przedziale od -6134 do 4484, czyli w teoretycznym zakresie $[-10000, 10000]$, co potwierdza występowanie utraconych aktualizacji. W wersji z zamkiem Petersona wszystkie 100 wykonań dało wynik 0, więc rozrzut wyników został całkowicie wyeliminowany.

4.2. Zadanie 2

Dla $N = 0$ teoretycznie najmniejsza, końcowa wartość licznika wynosi 0.
Dla $N = 1$ teoretycznie najmniejsza, końcowa wartość licznika wynosi 5.

Dla $N \geq 2$ teoretycznie najmniejsza wartość licznika po zakończeniu działania wszystkich wątków wynosi 2. Przykładowy przeplot operacji prowadzący do uzyskania tej wartości:

1. wątek A wykonuje `read` w pierwszej iteracji (odczytuje 0),
2. wątek B wykonuje 4 pełne iteracje (`read`, `inc`, `write`); wartość licznika wynosi 4;
3. pozostałe wątki wykonują w tym czasie wszystkie swoje operacje,
4. wątek A wykonuje `inc` i `write` – zapisuje 1 (nadpisując wszystkie dotychczasowe wyniki),
5. wątek B wykonuje `read` w ostatniej, piątej iteracji (odczytuje 1),
6. wątek A wykonuje pozostałe 4 iteracje; wartość licznika wynosi 5,
7. wątek B wykonuje `inc` i `write` – zapisuje 2,
8. program kończy działanie.

Końcowa wartość licznika wynosi 2.

Dowód minimalności wartości 2 dla $N \geq 2$:

1. Każda instrukcja `write` zapisuje wartość ≥ 1 (`read` + 1),
2. Zatem po pierwszej instrukcji `write` dowolnego wątku licznik ma już zawsze wartość ≥ 1,
3. Końcowa wartość pochodzi od ostatniej instrukcji `write` w całym wykonaniu. Należy ona do pewnego wątku X i jest jego 5. zapisem.
4. 5. odczyt wątku X następuje po jego własnym 1. zapisie, więc zawsze odczytuje wartość ≥ 1.
5. Ostatnia instrukcja `write` zapisuje wartość ≥ 2.

Razem z przeplotem z punktu 2.1, który osiąga wartość 2, oznacza to, że 2 jest najmniejszą możliwą wartością końcową.

5. Wnioski

Eksperyment z zadania 1 pokazuje, że brak synchronizacji prowadzi do nieprzewidywalnych wyników. Gdy wątki przeplatają kroki operacji `inc()` oraz `dec()`, jeden z nich może nadpisać wynik drugiego. Histogram wartości końcowych jest rozproszony w szerokim zakresie, co potwierdza, że wynik programu jest niedeterministyczny. Utracone aktualizacje mogą unieważnić efekt niemal wszystkich wykonanych wcześniej operacji.

Rozwiązanie w postaci zamka Petersona ma też ograniczenia. Przede wszystkim działa tylko dla dwóch wątków, a także stosuje aktywne oczekiwanie (*busy waiting*), zużywając czas procesora.

Programowanie współbieżne wymaga jawnego zadbania o poprawność dostępu do danych współdzielonych. Intuicja oparta na programach sekwencyjnych zawodzi, a błędy synchronizacji są trudne do wykrycia ze względu na swoją niedeterministyczną naturę. Rozwiązywanie problemu wyścigu wątków jest więc podstawą budowania niezawodnych systemów współbieżnych.

Bibliografia

[1] Wikipedia contributors, „Peterson's algorithm”. Dostęp: 7 października 2026. [Online]. Dostępne na: https://en.wikipedia.org/wiki/Peterson%27s_algorithm